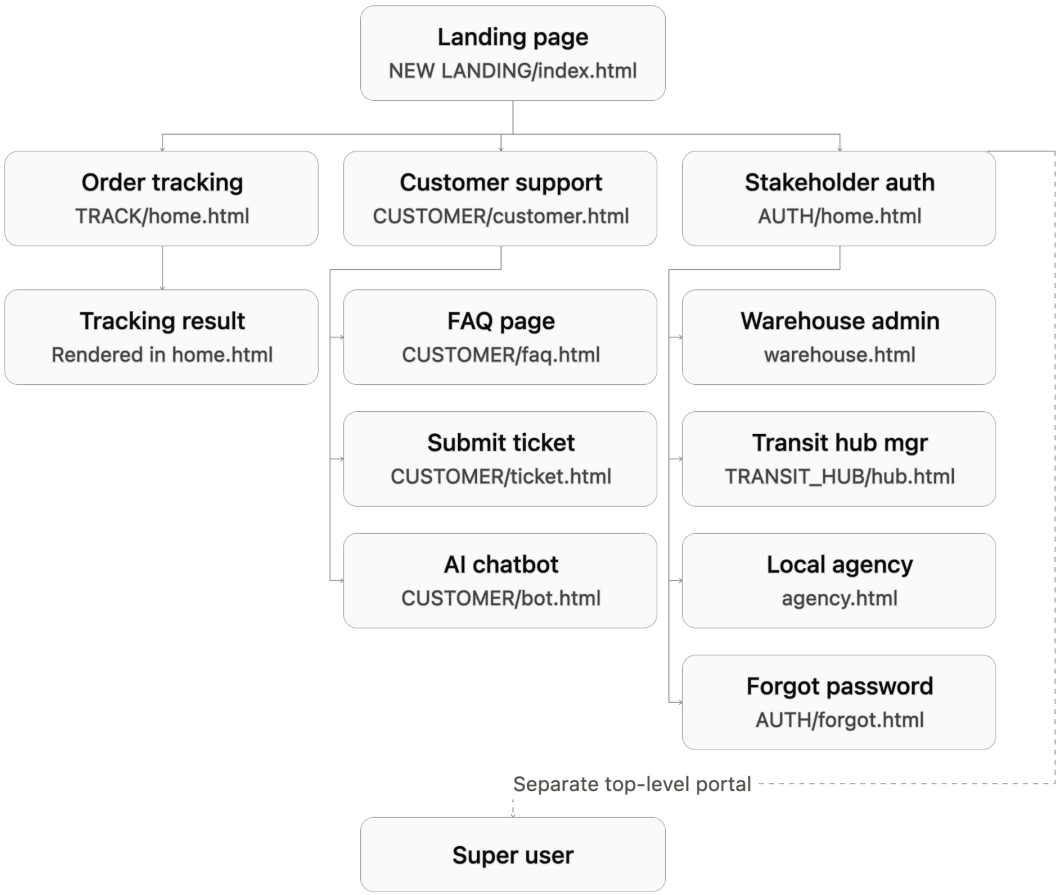
FDFED Lab-1

Existing Application Analysis & React Planning

Executive Summary

This document analyzes the existing **Ware2Door** logistics application, focusing on its workflows, reusable UI elements, dynamic behaviour, JavaScript, and NestJS APIs. It also identifies key frontend issues and areas where React can improve reusability and maintainability.The document concludes with a proposed React conversion target and an initial component structure.

Task 0 : Application Flow



Actor	Responsibilities
Customer (Public)	Views real-time package status, tracking history, interactive map location, and logs support tickets / AI support requests.
Warehouse Admin	Validates orders, manages stock inventory, registers manifest dispatches, and triggers pre-alerts.
Transit Hub Manager	Executes In-Scan/Out-Scan verification, manages sorting hub capacity, and verifies routing integrity.
Local Delivery Agency	Dispatches couriers, records delivery attempts, confirms drop-offs, and initiates Return to Origin (RTO) flows.
Super User	Global administrator monitoring platform KPIs, node capacity, audit logs, and operational overrides.

Where the User Enters vs Receives Data

Data entry points: Tracking search inputs (TRACK/home.html, CUSTOMER/bot.html); role selection & credentials (AUTH/home.html, SUPER_USER/su_auth.html); ticket creation forms and category dropdowns (CUSTOMER/ticket.html); stock addition, dispatch creation, and barcode scan inputs(WAREHOUSE/warehouse.html, TRANSIT_HUB/hub.html, LOCAL_AGENCY/agency.html).

Data display points: Embedded Google Maps view, visual step-by-step progress journeys, and event log tables; live inventory tables, hub capacity progress bars, and driver/vehicle assignment sheets; active RTO tracking
boards and ticket status logs.

Backend data flow: The frontend communicates with the NestJS backend at http://localhost:8000 via asynchronous fetch() requests, returning structured JSON for all live inventory, scan actions, tracking details, and support tickets

Task 1: Identification of Repeated UI

UI Element	Pages / Location	How is it repeated?	Why reuse is useful
Top Navigation Bar	NEW LANDING/index.html,TRACK/home.html, CUSTOMER/*.html	Identical HTML markup (logo, navigation links, login CTA) copied into each file.	Modifying a link or header style currently requires manual edits across 6+ separate HTML files.
Footer Component	NEW LANDING, TRACK, CUSTOMER/customer.html, CUSTOMER/faq.html	Identical footer sections with identical branding, links, and SVG social icons.	Brand updates or legal link updates can be made in a single reusable component.
Status Badge / Pill	TRACK/home.html, CUSTOMER/bot.html, WAREHOUSE/warehouse.html, LOCAL_AGENCY/agency.html	Custom-styled badges showing shipment status (Delivered, In Transit, RTO, Picked and Packed).	Standardizes color schemes (green, yellow, red) and prevents hardcoded inline styles.
Toast Alert Banner	TRACK/home.js, CUSTOMER/ticket.js, AUTH/script.js	Dynamically created fixed floating toast notifications with auto-dismiss timers.	Eliminates duplicate DOM creation logic (`document.createElement`) and synchronizes toast timeouts.
KPI Metric Card	WAREHOUSE/warehouse.html, TRANSIT_HUB/hub.html, SUPER_USER/superuser.html	Metric overview boxes (title, count/metric, icon, background gradient).	Allows passing dynamic props (title, count, icon, trendColor) instead of duplicating complex HTML grids.
Shipment Journey Timeline	TRACK/home.html, CUSTOMER/bot.html	Multi-step progress timeline indicating completed, active, and pending checkpoints.	Encapsulates step status calculation and icon rendering into a single declarative component.
Modal / Dialog Backdrop	CUSTOMER/bot.html, WAREHOUSE/warehouse.html, LOCAL_AGENCY/agency.html	Backdrop overlays with centered modal containers for tracking inputs or dispatch confirmations.	Centralizes focus trapping, escape-key handlers, and backdrop transitions.

Task 2: Identification of Dynamic Behaviour

Page / Feature	User Action	What Changes in the UI?
Track Order (TRACK/home.html)	Enters Tracking ID & clicks "Track" / hits Enter	Hides `.empty-state`, shows `.result-section`, updates map iframe with live lat/lng, renders timeline steps and event logs.
Track Order (TRACK/home.js)	Background polling timer fires (every 15s)	Re-fetches status; if changed, re-renders timeline, updates event table, triggers a floating status toast.
Login Portal (AUTH/home.html)	Clicks a Role Tab (Warehouse / Transit / Agency)	Removes `.active` class from previous tab, adds it to the selected tab, switches internal auth target state.
Login Portal (AUTH/home.html)	Clicks password eye icon	Toggles input type between password/text; toggles FontAwesome icon class (`fa-eye` vs `fa-eye-slash`).
Customer AI Bot (CUSTOMER/bot.html)	Enters Tracking ID in modal & clicks "Start Chat"	Sends validation request; on success fades out modal backdrop and displays chat window with shipment metadata.
Customer AI Bot (CUSTOMER/bot.html)	Clicks quick-prompt chip or types custom message	Appends user chat bubble, triggers animated typing indicator, appends AI response bubble with auto-scroll.
Ticket Portal (CUSTOMER/ticket.html)	Submits support ticket form	Dispatches POST payload; displays success/error banner with auto-dismiss timeout and resets form inputs.
Transit Hub (TRANSIT_HUB/hub.html)	Inputs barcode and clicks "In-Scan"	Validates hub assignment; moves package entry from expected inbound list to on-hand inventory; updates capacity meter.
Local Agency (LOCAL_AGENCY/agency.html)	Selects "Record Failed Attempt"	Opens modal prompt, captures failure reason, updates delivery row pill to RTO or Attempt Failed.

Task 3: Analysis of Existing JavaScript

JavaScript	What does it do?	Page / Feature	What UI does it affect?
`document.querySelector('.searcher')`	Captures DOM reference to user tracking input	TRACK/home.js	Reads tracking ID string entered by user.
`document.querySelector('.empty-state').style.display = 'none'`	Imperatively hides empty container	TRACK/home.js	Hides empty placeholder illustration when results load.
`container.innerHTML = ...` (template strings)	Injects dynamic HTML markup strings into container	TRACK/home.js, CUSTOMER/bot.js, TRANSIT_HUB/hub.js	Overwrites shipment summary, timeline steps, logs, and chat messages.
`classList.toggle('scrolled', window.scrollY > 50)`	Adds/removes CSS class on window scroll	NEW LANDING, CUSTOMER/ticket.js	Toggles navbar backdrop blur and background styling.
`document.createElement('div') + appendChild()`	Dynamically generates toast/row elements in DOM	TRACK/home.js, CUSTOMER/bot.js	Mounts floating toast notification banner and chat bubbles.
`setInterval(async () => {...}, 15000)`	Starts unmanaged polling loop	TRACK/home.js	Repeatedly triggers network request and updates tracking UI.
`localStorage.getItem('TID') / setItem(...)`	Reads/writes local browser cache	TRACK/home.js, CUSTOMER/customer.js	Pre-fills input on page load and enables auto-tracking across redirects.
`role_opt.forEach(opt => opt.className = 'role-option')`	Loops over elements to reset class attribute	AUTH/script.js	Clears active highlights on role selection buttons.

Task 4: Analysis of NestJS API

Feature	Method	API Endpoint	Purpose
Authentication	POST	`/auth/warehouse`	Authenticates warehouse admin credentials.
Authentication	POST	`/auth/transit`	Authenticates transit hub manager credentials.
Authentication	POST	`/auth/agency`	Authenticates local delivery agency credentials.
Public Tracking	GET	`/track/:id`	Returns complete tracking timeline, coordinates, and status history for a Tracking ID.
Customer Support	POST	`/tickets/raise`	Creates a new customer support ticket.
Customer Support	GET	`/tickets/validate/:trackingId`	Validates tracking ID existence before ticket creation or AI support session.
Shipments (Warehouse)	POST	`/:warehouseId/dispatch`	Dispatches pending manifest orders to designated transit hub and agency.
Inventory (Warehouse)	GET	`/:warehouseId/stockInventory`	Fetches real-time stock levels of warehouse items.
Transit Hub Scan	POST	`/hub/:hubId/inscan`	Registers physical package arrival and updates status to In Scan at Transit Hub.
Transit Hub Scan	POST	`/hub/:hubId/outscan`	Registers package departure toward the destination delivery agency.
Agency Delivery	POST	`/agency/:agencyId/deliveries/:id/deliver`	Marks package as successfully delivered to customer.
Return to Origin (RTO)	POST	`/agency/:agencyId/rto`	Initiates reverse logistics flow for failed deliveries.
Super User Admin	GET	`/su/overview`	Returns aggregate counts and system-wide statistics.

Task 5: Problems in the Existing Frontend

Current Issue	Where Does It Occur?	Why Is It a Problem?
1. Fragile imperative DOM injection (innerHTML)	TRACK/home.js, CUSTOMER/bot.js, WAREHOUSE/warehouse.js	Constructing large UI trees via raw template-string concatenation is vulnerable to XSS, breaks attached event listeners, and causes unneeded DOM re-renders.
2. Heavy markup duplication across pages	NEW LANDING, TRACK, CUSTOMER/*.html, AUTH/*.html	Headers, navigation bars, modal dialogs, and footers are manually copy-pasted across 8+ HTML files; any change requires synchronizing every file manually.
3. Scattered global & DOM-coupled state	AUTH/script.js, TRACK/home.js, CUSTOMER/bot.js	State is tracked via global variables (`let state = "WareHouse"`) and by reading CSS classes from the DOM, creating synchronization bugs.
4. Unmanaged timers & risk of memory leaks	TRACK/home.js (setInterval polling)	Polling intervals and window scroll listeners are started without structured lifecycle cleanup, leaking memory on navigation.
5. Lack of client-side SPA routing	Entire application	Navigating between Landing, Tracking, Support, and Dashboard causes full browser refreshes, losing transient state such as chat history or active filters.
6. Duplicated network & error-handling logic	Across all `.js` files	Every script independently implements boilerplate `fetch()`, header creation, toast display, and JSON parsing without a unified API client or interceptor.

React conversion opportunities

In our current project, we write raw HTML and use plain JavaScript to manually find elements on the screen (document.getElementById) and overwrite them with long text strings (innerHTML).

React allows us to change this approach in 4 major ways:

A. Reusable "Lego Blocks" (Instead of Copy-Pasting Code)
Current Problem: We have copy-pasted the exact same navigation bar and footer into 8 different HTML files. If we want to change a single link, we have to open all 8 files and update them manually.
With React: We create one template for the Navigation Bar and one template for the Footer. We can then insert them into any page with a single line of code. Changing it once updates all pages automatically.

B. Automatic Screen Updates (No more innerHTML Mess)
Current Problem: When tracking data comes from the server, we write 50+ lines of JavaScript to stitch HTML strings together and push them into the page using innerHTML. If there is a small typo or missing quote, the whole screen breaks.
With React: We simply tell the page: "Here is the data we received from the backend." React automatically updates the text, colors, and progress bars on the screen without us manually editing the HTML.

C. Smoother Page Transitions (No White Flashes / Reloads)
Current Problem: Clicking a link to go from the Landing page to the Tracking or Support page causes the entire browser to reload, creating a blank white flash and resetting everything.
With React: React can swap only the middle content of the page instantly while keeping the website smooth and fast, just like a mobile app.

D. Single Central Memory (Instead of scattered variables)
Current Problem: Information like the user's role, login status, and active tracking ID is scattered across different script files and browser storage.
With React: All important data is kept in a clean, organized "memory" for each page so everything stays in sync.

First React Conversion Target

Selected Page: The Order Tracking Page (TRACK/home.html + home.js)

Why this page is the best choice to convert first:

- **It's Public & Simple to Test**: It doesn't require logging in with special passwords or roles. Anyone can open it, type a tracking ID, and see results.
- **Clear Step-by-Step Flow:**
  User types Tracking ID → App fetches data from backend → Screen displays the result.
- **Solves the Messiest JavaScript Code:**
  Currently, TRACK/home.js has a 15-second background timer and lots of manual DOM manipulation. Converting this to React will simplify the code significantly.
- **Reusable Parts for Other Pages:**
  The progress steps, status badges, and map box built for this page can be reused later in the Warehouse, Transit Hub, and Agency dashboards.

Initial React conversion sketch